Лекция 4
Основы унифицированного языка моделирования

UML (Unified Model Language)

ОСНОВНЫЕ ДИАГРАММЫ

Унифицированный язык объектно-ориентированного моделирования, синтаксис и семантика основных объектов UML, понятия: модель, диаграмма. Диаграмма прецедентов

UML (англ. *Unified Modeling Language* — унифицированный язык моделирования)

- **Язык UML** представляет собой общецелевой язык визуального моделирования, который разработан для спецификации, визуализации, проектирования и документирования компонентов программного обеспечения, бизнес-процессов и других систем. Язык UML может быть эффективно использован для построения концептуальных, логических и графических моделей сложных систем самого различного целевого назначения. Язык UML предлагает набор инструментальных средств, позволяющих проводить всесторонний анализ сложных ИС как с технической точки зрения, так и с точки зрения потребностей бизнеса.

UML упрощает процесс проектирования ИС, снижает его стоимость и повышает эффективность.

UML (Unified Model Language) – семейство графических нотаций в основе которого лежит единая метамодель.

Создан, чтобы помогать в описании и проектировании информационных систем любой сложности.

Считается графическим языком моделирования.

Используется в качестве инструмента обратного и прямого инжиниринга в CASE-средствах разработки информационных систем.

Группа OMG и стандартизация UML

OMG – организация, занимающаяся созданием методологии на основе существующих стандартов.

С момента появления первой версии UML, методологии UML создавались в OMG.



OMG Консорциум, занимающийся разработкой и продвижением объектно-ориентированных технологий и стандартов.

Это некоммерческое объединение, разрабатывающее стандарты для создания интероперабельных, то есть платформо - независимых, приложений на уровне предприятия.

С консорциумом сотрудничает около 800 организаций - крупнейших производителей программного обеспечения.

Существует три основных ***режима*** использования UML диаграмм:

- режим эскиза
- режим проектирования
- режим языка программирования

У режимов есть две опции:

прямой (диаграммы до кода)

и обратный (диаграммы на основании кода) ***инжиниринг***.

МОДЕЛЬ UML И ЕЕ ЭЛЕМЕНТЫ

Модель UML (UML model) — это совокупность конечного множества конструкций языка, главные из которых — это сущности и отношения между ними.

Сами сущности и отношения модели являются экземплярами метаклассов метамодели.

Сущности

Для удобства обзора сущности в UML можно подразделить на четыре группы:

- структурные;
- поведенческие;
- группирующие;
- аннотационные.

Основные понятия языка UML.

- **Объект**— сущность реального мира или концептуальная сущность, либо программная единица, состоящая из атрибутов (полей) и методов для их обработки. Характеризуется состоянием, поведением, и уникальностью. Состояние определяется атрибутами, а поведение – методами.
- **Класс** – описание множества объектов с общими атрибутами, определяющими состояние, и операциями, определяющими поведение. Класс изображается прямоугольником, разделенный на три секции
  В первом приводится имя класса, во втором – список атрибутов, в третьем – список методов. **Ассоциация** – наиболее общий тип отношений первого рода, связанных с передачей сообщений ассоциированным классам путем делегирования

Структурные сущности UML

Структурные сущности, предназначены для описания структуры, к ним обычно относят:

Объект (object) — сущность, обладающая уникальностью и инкапсулирующая в себе состояние и поведение.

Класс (class) — описание множества объектов с общими атрибутами, определяющими состояние, и операциями, определяющими поведение.

Интерфейс (interface) — именованное множество операций, определяющее набор услуг, которые могут быть запрошены потребителем и предоставлены поставщиком услуг.

Кооперация (collaboration) — совокупность объектов, которые взаимодействуют для достижения некоторой цели.

Действующее лицо (actor) — сущность, находящаяся вне моделируемой системы и непосредственно взаимодействующая с ней.

Компонент (component) — модульная часть системы с четко определенным набором требуемых и предоставляемых интерфейсов.

Артефакт (artifact) — элемент информации, который используется или порождается в процессе разработки программного обеспечения. Иначе, артефакт — это физическая единица реализации, получаемая из элемента модели (например, класса или компонента).

Узел (node) — вычислительный ресурс, на котором размещаются и при необходимости выполняются артефакты.

Поведенческие сущности UML

Поведенческие сущности предназначены для описания поведения.

Основных поведенческих сущностей всего две: состояние и действие

(точнее, две с половиной, потому что иногда употребляется еще и деятельность, которую можно рассматривать как особый случай состояния).

Состояние (state) — период в жизненном цикле объекта, находясь в котором объект удовлетворяет некоторому условию и осуществляет собственную деятельность или ожидает наступления некоторого события.

Деятельность (activity) можно считать частным случаем состояния, который характеризуется Продолжительными (по времени) не атомарными вычислениями.

Действие (action) — примитивное атомарное вычисление.

Это только очевидная часть айсберга поведенческих сущностей: состояния бывают разные.

Также, при моделировании поведения используется еще ряд вспомогательных сущностей, которые сосуществуют только вместе с указанными основными.

Несколько особняком стоит сущность — вариант использования (use case), которой присущи Как структурные, так и поведенческие аспекты.

Группирующие и аннотационные сущности UML

Вариант использования (use case) — множество сценариев, объединенных по некоторому Критерию и описывающих последовательности производимых системой действий, доставляющих значимый для некоторого действующего лица результат.

Приведенная классификация не является исчерпывающей.

У каждой из этих сущностей есть различные частные случаи и вариации

Группирующая сущность в UML одна — пакет — универсальная.

Пакет (package) — группа элементов модели (в том числе пакетов).

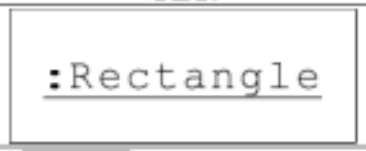
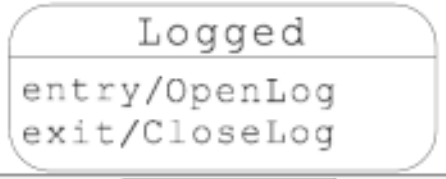
Аннотационная сущность тоже одна — примечание (comment) — зато в нее можно поместить все что угодно, так как содержание примечания UML не ограничивает.

Нотация основных сущностей UML

Нотация основных сущностей

Название	Графическая нотация
Артефакт	«artifact» Requirement Specification; «library» QT
Вариант использования	Make Order; Make Order
Действующее лицо	Customer; Customer
Деятельность и действие	Display main menu; a=15
Интерфейс	IAudio; «interface» IAudio
Класс	Product; Order {abstract}
Компонент	DataBase; «component» DataBase

Нотация основных сущностей UML

Название	Графическая нотация
Кооперация	
Объект	
Пакет	
Примечание	
Состояние	
Узел	

Отношения в UML

В UML используются четыре основных типа отношений:

- зависимость (dependency);

- ассоциация (association);

- обобщение (generalization);

- реализация (realization).

Перечисленные типы отношений являются основными, но возможны также различные их вариации и дополнительные отношения.

Отношение зависимости в UML

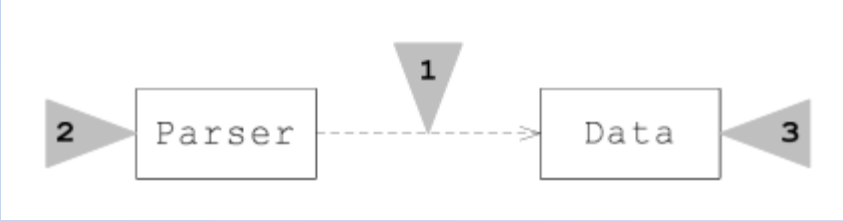
Зависимость — это наиболее общий тип отношения между двумя сущностями.

Отношение зависимости указывает на то, что изменение независимой сущности каким-то образом влияет на зависимую сущность.

Графически отношение зависимости изображается в виде **пунктирной линии** со стрелкой (1), направленной от зависимой сущности (2) к независимой (3).

Как правило, семантика Конкретной зависимости уточняется в модели с помощью дополнительной информации.

Например, зависимость со стереотипом «use» означает, что зависимая сущность использует (скажем, вызывает операцию) независимую сущность.



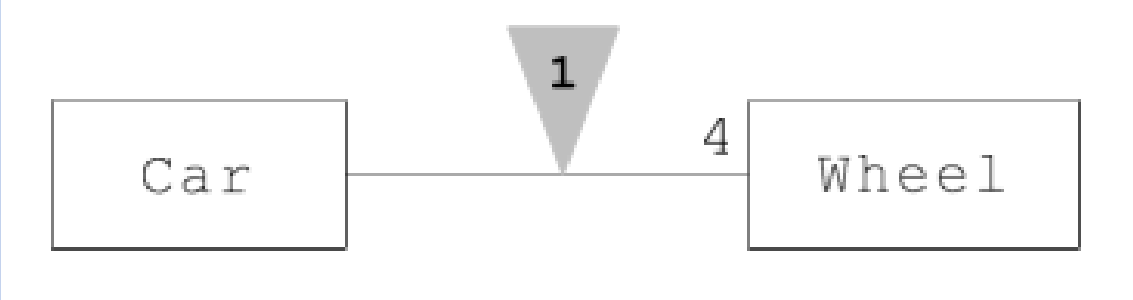
Отношение зависимости в UML

Ассоциация — это наиболее часто используемый тип отношения между сущностями. Отношение ассоциации имеет место, если одна сущность непосредственно связана с другой (или с другими — ассоциация может быть не только бинарной).

Графически ассоциация изображается в виде **сплошной линии** с различными дополнениями, соединяющей связанные сущности.

На программном уровне непосредственная связь может быть реализована различным образом, главное, что ассоциированные сущности знают друг о друге.

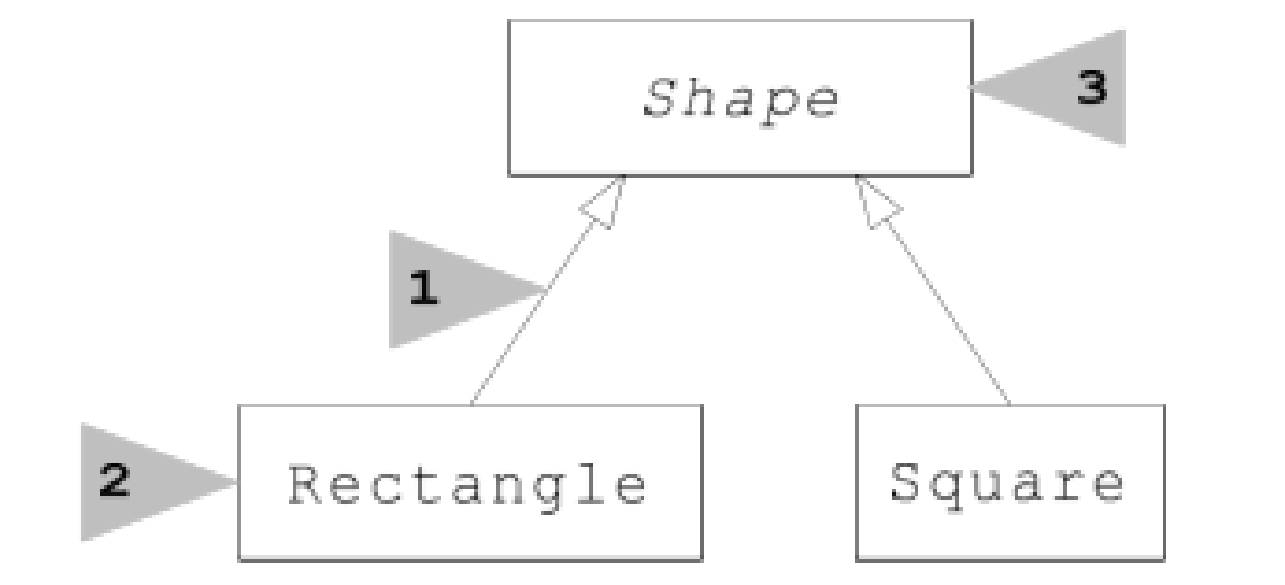
Например, отношение часть–целое является частным случаем ассоциации и называется отношением агрегации.



Отношение обобщения в UML

Обобщение — это отношение между двумя сущностями, одна их которых является частным (специализированным) случаем другой.

Графически обобщение изображается в виде **линии с треугольной незакрашенной стрелкой на конце** (1), направленной от частного (2) (подкласса) к общему (3) (суперклассу)

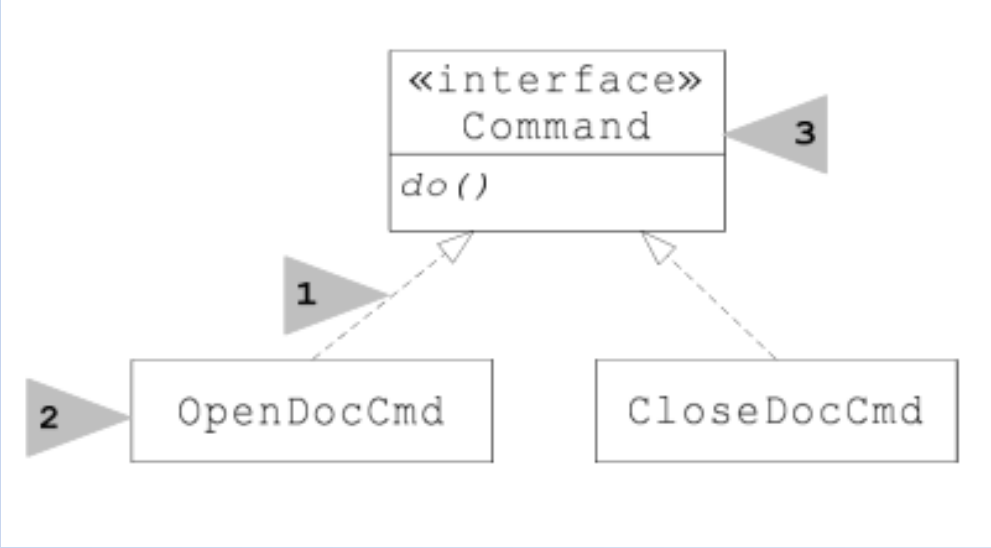


Отношение реализации в UML

Отношение **реализации** используется несколько реже, чем предыдущие три типа отношений, поскольку часто подразумеваются по умолчанию.

Отношение реализации указывает, что одна сущность является реализацией другой.

Графически реализация изображается в виде **пунктирной линии с треугольной незакрашенной стрелкой на конце** (1), направленной от реализующей сущности (2) к реализуемой (3)



Диаграммы в UML

Диаграммы UML есть та основная накладываемая на модель структура, которая облегчает создание и использование модели.

Диаграмм (diagram) — это графическое представление некоторой части графа модели.

Вообще говоря, в диаграмму можно было бы включить любые (допустимые) комбинации сущностей и отношений, но произвол в этом вопросе затруднил бы понимание моделей.

Поэтому авторы UML определили набор рекомендуемых к использованию типов диаграмм, которые получили название канонических типов диаграмм.

Заметим, что помимо сущностей и отношений на диаграмме присутствует другие элементы модели, которые мы также будем называть конструкциями языка.

Это тексты, которые могут быть написаны внутри фигур сущностей или рядом с линиями отношений, рамки диаграмм и их фрагментов, значки, присоединяемые к линиям или помещаемые внутрь фигур.

Эти элементы не только помогают представить модель в более наглядной форме, но подчас несут значительную смысловую нагрузку.

Классификация диаграмм в UML

В UML 1 всего определено 9 канонических типов диаграмм.

Ниже перечислены их названия, принятые в данном учебном пособии

(в других источниках есть отличия).

- Диаграмма использования (Use Case diagram).
- Диаграмма классов (Class diagram).
- Диаграмма объектов (Object diagram).
- Диаграмма состояний (State chart diagram).
- Диаграмма деятельности (Activity diagram).
- Диаграмма последовательности (Sequence diagram).
- Диаграмма кооперации (Collaboration diagram).
- Диаграмма компонентов (Component diagram).
- Диаграмма размещения (Deployment diagram).

Классификация диаграмм в UML

В UML 2 внесены значительные коррективы как в список канонических диаграмм, а именно их число увеличилось до 13, так и в список доступных конструкций языка, что значительно расширило область его применения.

Кроме этого две диаграммы были переименованы: диаграмма кооперации была переименована в диаграмму коммуникации, а диаграмма состояний в диаграмму автомата.

Список новых диаграмм и их названий, приведен ниже. На рисунках ниже приведены соответствующие диаграммы классов.

- Диаграмма внутренней структуры (Composite Structure diagram).

- Диаграмма пакетов (Package diagram).

- Диаграмма автомата (State machine diagram).

- Диаграмма коммуникации (Communication diagram).

- Обзорная диаграмма взаимодействия (Interaction Overview diagram).

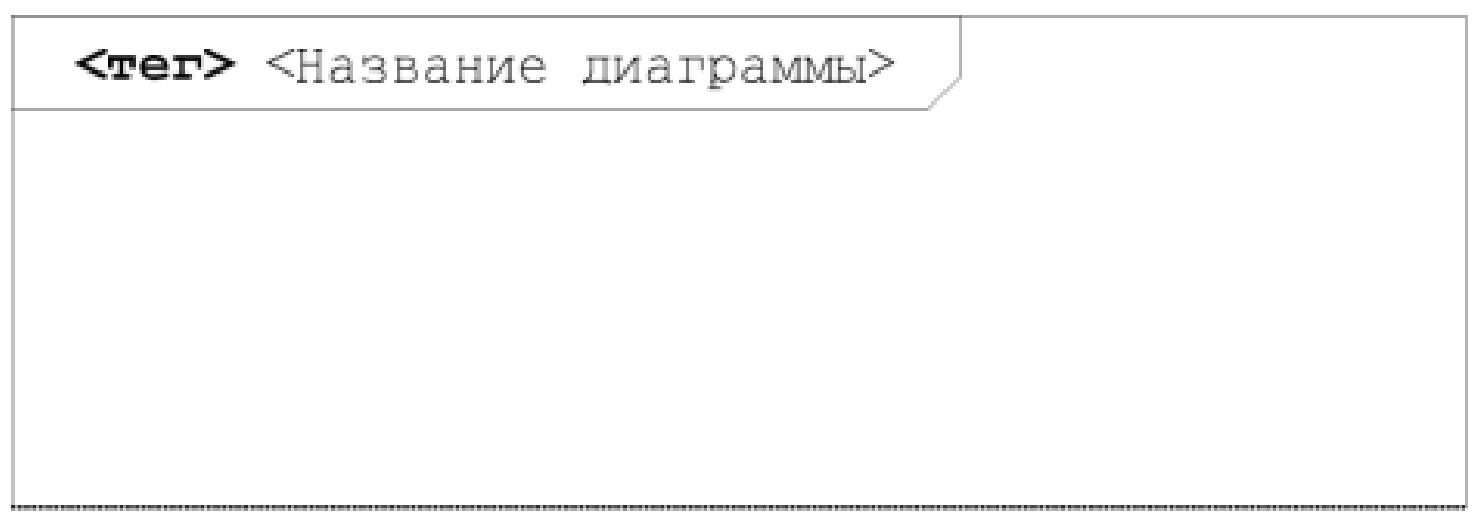
- Диаграмма синхронизации (Timing diagram).

Рассмотрим как стандарт требует оформлять диаграммы.

Оформление диаграмм в UML

Основных элементов оформления два: наружная рамка и ярлычок с названием диаграммы. Если с рамкой все просто – это прямоугольник, ограничивающий область в котором должны находиться элементы диаграммы, то название Диаграммы записывается в специальном формате, приведенном на рисунке Возможные теги (типы) для диаграмм приведены в таблице Теги, предлагаемые стандартом, записаны во второй столбец.

Однако, как показала практика, предлагаемые стандартом правила не всегда удобны и логически обоснованы, поэтому третий столбец таблицы содержит взятую из сложившейся практики альтернативу.



Оформление диаграмм в UML

Типы диаграмм для заголовка

Название диаграммы	Тег (стандартный)	Тег (предлагаемый)
Диаграмма использования	use case или uc	use case
Диаграмма классов	class	class
Диаграмма автомата	state machine или stm	state machine
Диаграмма деятельности	activity или act	activity
Диаграмма последовательности	interaction или sd	sd
Диаграмма коммуникации	interaction или sd	comm
Диаграмма компонентов	component или cmp	component
Диаграмма размещения	не определен	deployment
Диаграмма объектов	не определен	object
Диаграмма внутренней структуры	class	class или component
Обзорная диаграмма взаимодействия	interaction или sd	interaction
Диаграмма синхронизации	interaction или sd	timing
Диаграмма пакетов	package или pkg	package

Классификация диаграмм в UML

Все диаграммы UML можно условно разбить на две группы.

Первая из которых — общие диаграммы, вторая специальные диаграммы

Общие диаграммы

практически не зависят от предмета моделирования и могут применяться в любом программном проекте без оглядки на предметную область, область решений и т. д.

Специальные диаграммы

Специальные диаграммы характеризуются тем, что чаще всего служат для дополнения какой-либо общей диаграммы, например, являются ее частным случаем или же просто играют вспомогательную роль, уточняя некоторые детали.

Типы диаграмм UML

- Структурные
  - Классов(StaticStructurediagram, Classdiagram)
  - Объектов (Objectdiagram)
  - Компонентов (Component diagram)
  - Развёртывания (Deployment diagram)
- Поведения
  - Прецедентов (Use case diagram, Вариантов использования)
  - Состояний (State diagram)
  - Активности (Activity diagram, Деятельности)
- Взаимодействия
  - Кооперации (Колаборации, Коммуникации, Collaboration diagram) – подвид диаграммы композитной/составной структуры
  - Последовательности (Sequence diagram)

Основные типы диаграмм нотации UML

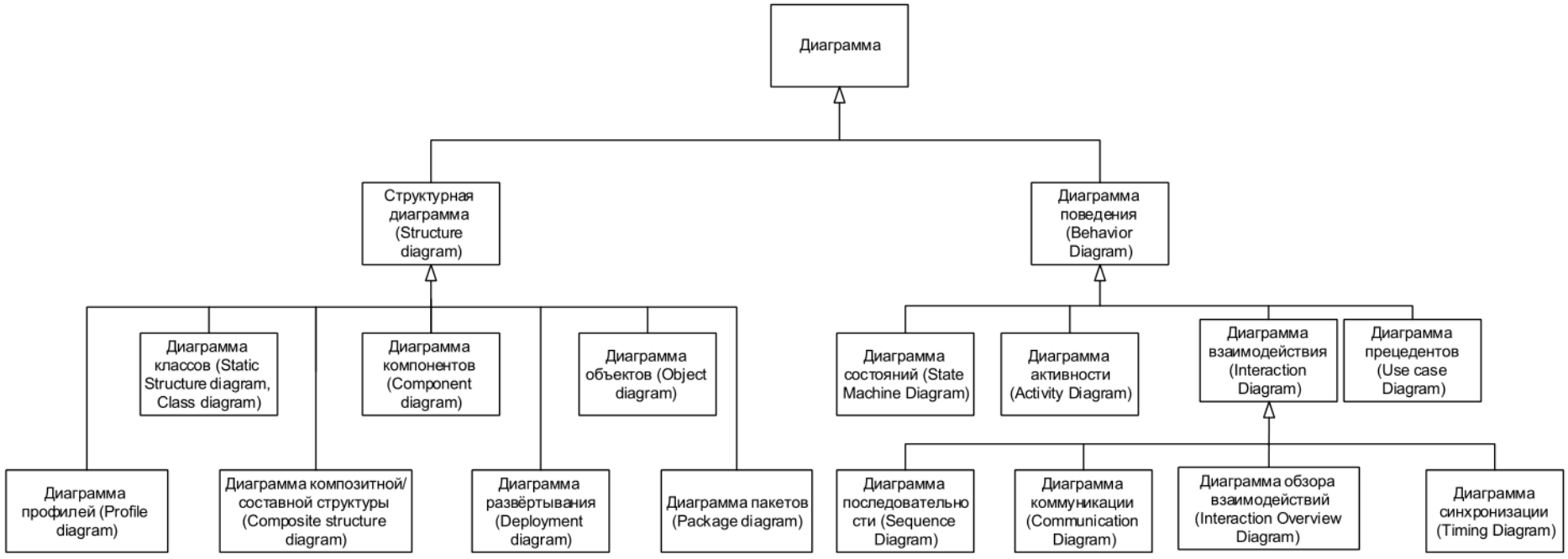


Диаграмма прецедентов

- Диаграммы прецедентов применяются для моделирования вида системы с точки зрения внешнего наблюдателя. На диаграмме прецедентов графически показана совокупность прецедентов и субъектов, а также отношения между ними. **Прецедент** (use case) —это описание множества последовательностей действий (включая их варианты), которые выполняются системой для того, чтобы актер получил результат, имеющий для него определенное значение. Стандартным графическим обозначением прецедента на диаграммах является эллипс, внутри которого содержится краткое название прецедента или имя в форме глагола с пояснительными словами.

Прецеденты – это технология определения функциональных требований к системе

Работа прецедентов заключается в описании типичных взаимодействий между пользователями системы и самой системойи предоставлении описания процесса ее функционирования.

В терминах прецедента пользователи называются актерами. Актер (actor) представляет собой некую роль, которую пользователь играет по отношению к системе.

Примерами актера могут быть пользователь, торговый представитель пользователя, менеджер по продажам и товаровед.

Актеры действуют в рамках прецедентов.

Один актер может выполнять несколько прецедентов; и наоборот, в соответствии с одним прецедентом могут действовать несколько актеров.

Обычно клиентов много, поэтому роль клиента могут играть многие люди.

К тому же один человек может играть несколько ролей, например менеджер по продажам, выполняющий роль торгового представителя клиента.

Актер не обязательно должен быть человеком. -

Если система предоставляет некоторый сервис другой компьютерной системе, то другая система является актером.

Описание прецедента

В каждом прецеденте есть ведущий актер, который посылает системе запрос на обслуживание.

Ведущий актер – это актер, желание которого пытается удовлетворить прецедент и который обычно, но не всегда, является инициатором прецедента.

Одновременно могут быть и другие актеры, с которыми система также взаимодействует во время выполнения прецедента.

Они называются второстепенными актерами.

Каждый шаг в прецеденте – это элемент взаимодействия актера с системой.

Каждый шаг должен быть простым утверждением и должен четко указывать, кто выполняет этот шаг.

Шаг должен показывать намерение актера, а не механику его действий.

Следовательно, в прецеденте интерфейс актера не описывается.

Действительно, составление прецедента обычно предшествует разработке интерфейса пользователя.

Текст прецедента

В тексте описывается главный успешный сценарий прецедента.

После этого приводятся расширения (ответвления или вариации) прецедента.

Главный успешный сценарий:

1. Покупатель просматривает каталог и выбирает товары для покупки.
2. Покупатель оценивает стоимость всех товаров.
3. Покупатель вводит информацию, необходимую для доставки товара (адрес, доставка на следующий день или в течение трех дней).
4. Система предоставляет полную информацию о цене товара и его доставке.
5. Покупатель вводит информацию о кредитной карточке.
6. Система осуществляет авторизацию счета покупателя.
7. Система подтверждает оплату товаров немедленно.
8. Система посылает подтверждение оплаты товаров по адресу электронной почты покупателя.

Расширения:

3а. Клиент является постоянным покупателем.

.1: Система предоставляет информацию о текущей покупке и ее цене, а также информацию о счете.

.2: Покупатель может согласиться или изменить значения по умолчанию, затем возвращаемся к шагу 6 главного успешного сценария.

6а. Система не подтверждает авторизацию счета.

.1: Пользователь может повторить ввод информации о кредитной карте или закончить сеанс.

Расширение прецедента

Расширение внутри прецедента указывает условие, которое приводит к взаимодействиям, отличным от описанных в главном успешном сценарии (main success scenario, MSS), и устанавливает, в чем состоят эти отличия.

Расширение начинается с имени шага, на котором определяется это условие, и предоставляет краткое описание этого условия.

Следуйте этому условию, нумеруя шаги таким же образом, что и в главном успешном сценарии. Заканчивайте эти шаги описанием точки возврата в главный успешный сценарий, если это необходимо.

Структура прецедента – это отличный инструмент для поиска альтернатив главного успешного сценария.

На каждом шаге спрашивайте:

«Что может еще произойти?» и в частности «Что может пойти не так?»

Обычно лучше сначала изучить все возможные условия расширения, чтобы потом не увязнуть в трясине работы над последствиями.

Расширение прецедента

Сложный шаг в прецеденте можно представить другим прецедентом.

В терминах языка UML мы говорим, что первый прецедент включает (includes) второй.

Не существует стандартного способа показать в текст включение прецедента,

Но возможно использовать подчеркивание, которое пред полагает гиперссылку, и во многих инструментах действительно будет гиперссылкой.

Включенные прецеденты могут быть полезными в случае сложных шагов, которые иначе загромождали бы главный сценарий, или когда одни и те же шаги присутствуют в нескольких сценариях.

Не стоит разбивать прецеденты на под прецеденты и использовать их для функциональной декомпозиции.

Расширение прецедента

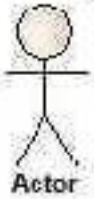
Наряду с шагами сценария можно вставить в прецедент дополнительную общую информацию.

- Предусловие (pre-condition) описывает действия, обязательно выполняемые системой перед тем, как она разрешит начать работу прецедента.
  Это полезная информация, позволяющая разработчикам не проверять некоторые условия в их программе.
- Гарантия (guarantee) описывает обязательные действия системы по окончании работы шаблона ответа. Успешные гарантии выполняются после успешного сценария; минимальные гарантии выполняются после любого сценария.
- Триггер (trigger) определяет событие, инициирующее выполнение прецедента.

Диаграмма прецедентов (Use Case Diagram)

Структурные элементы

Актер



Вариант использования

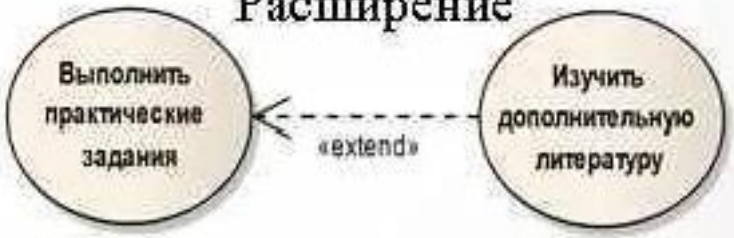


Взаимосвязи

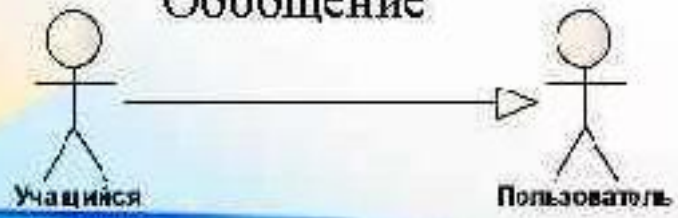
Ассоциация



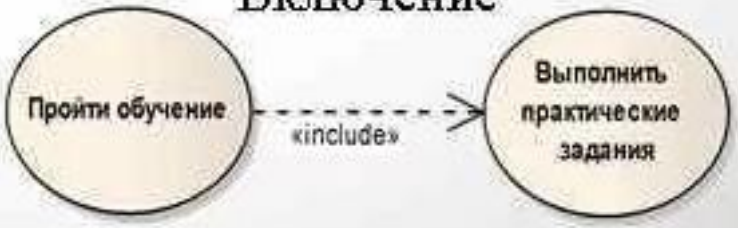
Расширение



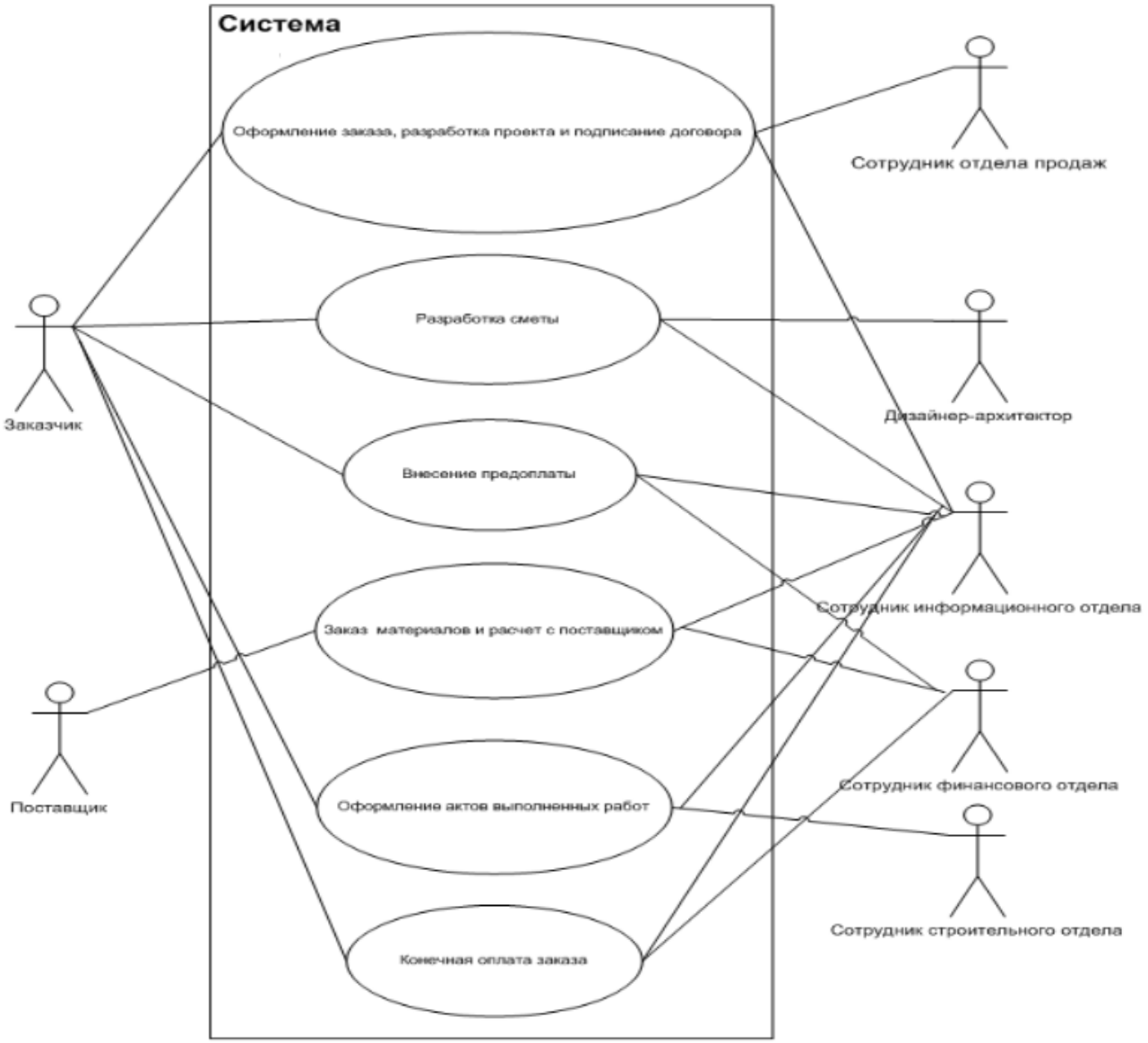
Обобщение



Включение



Пример диаграммы прецедентов



Диаграммы прецедента

Лучше всего обдумывать диаграмму прецедентов с помощью графической таблицы, показывающей их содержимое.

Она напоминает диаграмму контекста, используемую в структурных методах, поскольку она показывает границы системы и ее взаимодействие с внешним миром.

Диаграмма прецедентов показывает актеров, прецеденты и отношения между ними:

- Какие актеры выполняют тот или иной прецедент
- Какие прецеденты включают другие прецеденты

В языке UML помимо отношения «include» (включает) есть и другие типы отношений между прецедентами, например отношение «extend» (расширяет). Не следует слишком злоупотреблять ими лучше описывать сам прецедент

Уровни прецедентов

Общая проблема прецедентов состоит в том, что, увлекшись взаимодействием пользователя с системой, можно не обратить внимание на тот факт, что лучшим способом решения проблемы может быть изменение самого бизнес-процесса.

Часто можно слышать упоминание о прецедентах системы и прецедентах бизнес-процессов.

Эта терминология не является точной, но обычно считается, что прецедент системы (system use case) описывает особенности взаимодействия с программным обеспечением, тогда как прецедент бизнес-процесса (business use case) представляет собой реакцию бизнес-процесса на действие клиента или некоторое событие.

Уровни прецедентов

В книге Кокборна предлагается схема уровней прецедентов.

Базовый прецедент находится на «уровне моря».

Прецеденты уровня моря (sea level) обычно представляют отдельное взаимодействие ведущего актера и системы.

Такие прецеденты предоставляют ведущему актеру какой-либо полезный результат и обычно занимают от пары минут до получаса.

Прецеденты, которые существуют в системе, только если они включены в прецеденты уровня моря, называются прецедентами уровня рыб (fish level).

Прецеденты высшего уровня, уровня воздушного змея (kite-level), показывают, как прецеденты уровня моря на страиваются на более широкое взаимодействие с бизнес-процессами.

Обычно прецеденты уровня воздушного змея являются прецедентами Бизнес-процессов, а на уровне моря и на уровне рыб находятся прецеденты системы. Большинство ваших прецедентов должно принадлежать уровню моря.

Уровни прецедентов

В экстремальном программировании (Extreme Programming) возможности системы называются пожеланиями пользователя.

Общим является вопрос о том, как установить соответствие между возможностями и прецедентами.

Использование возможностей – это хороший способ разделения системы на блоки при планировании итеративного процесса, в результате чего каждая итерация предоставляет определенное количество возможностей.

Таким оразом оба приема описывают требования, их цели различны.

Описать возможности можно сразу, но многие специалисты считают более удобным в первую очередь разработать прецеденты, а уже затем сгенерировать список возможностей.

Возможность может быть представлена целым прецедентом, сценарием в прецеденте, шагом в прецеденте или каким-либо вариантом поведения

Обычно возможности получаются более четко определенными, чем прецеденты.

Применение прецедентов

Прецеденты представляют собой ценный инструмент для понимания функциональных требований к системе.

Первый вариант прецедентов должен составляться на ранней стадии выполнения проекта.

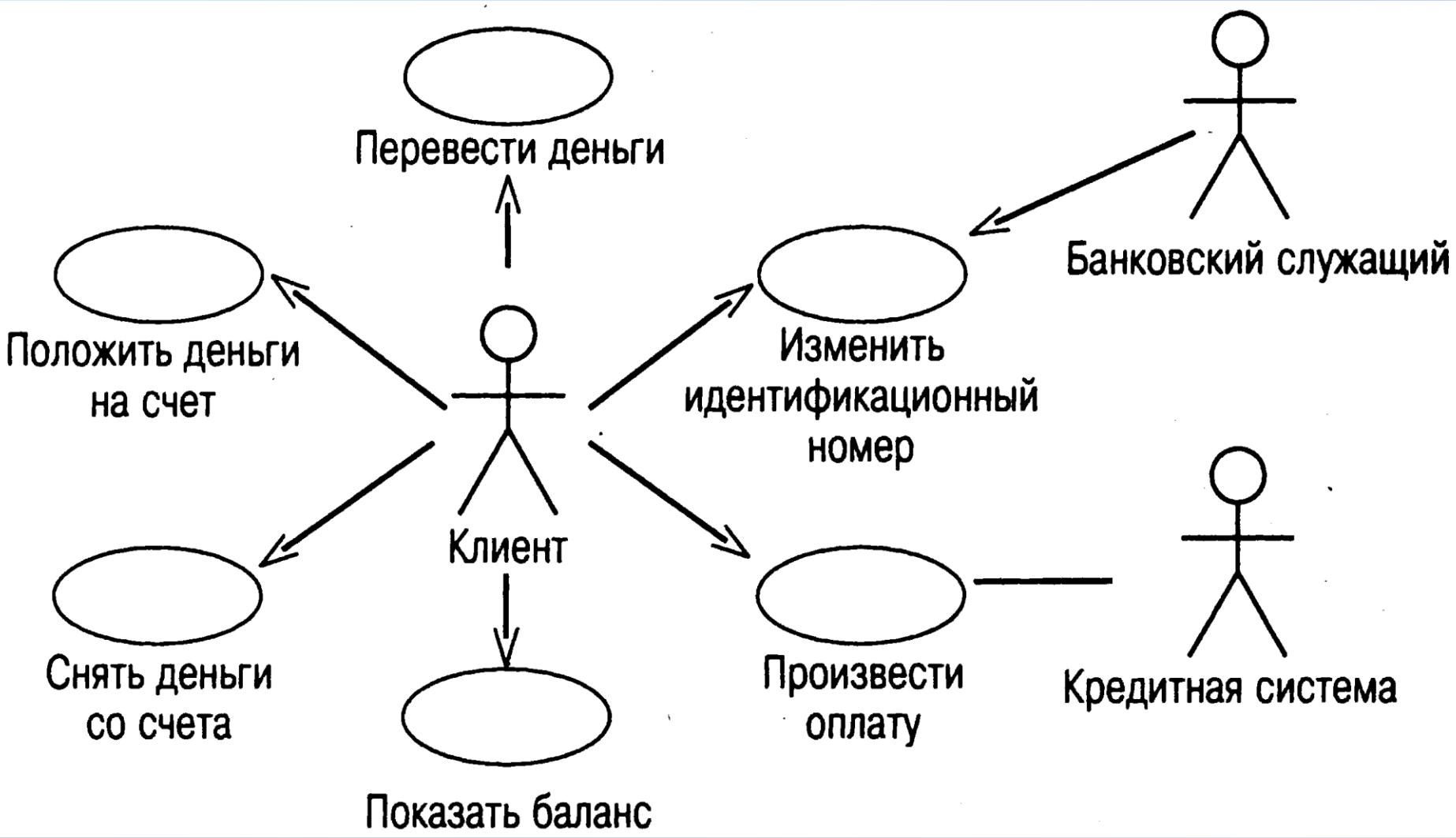
Более подробные версии прецедентов должны появляться непосредственно перед реализацией данного прецедента.

Важно помнить, что прецеденты представляют взгляд на систему со стороны.

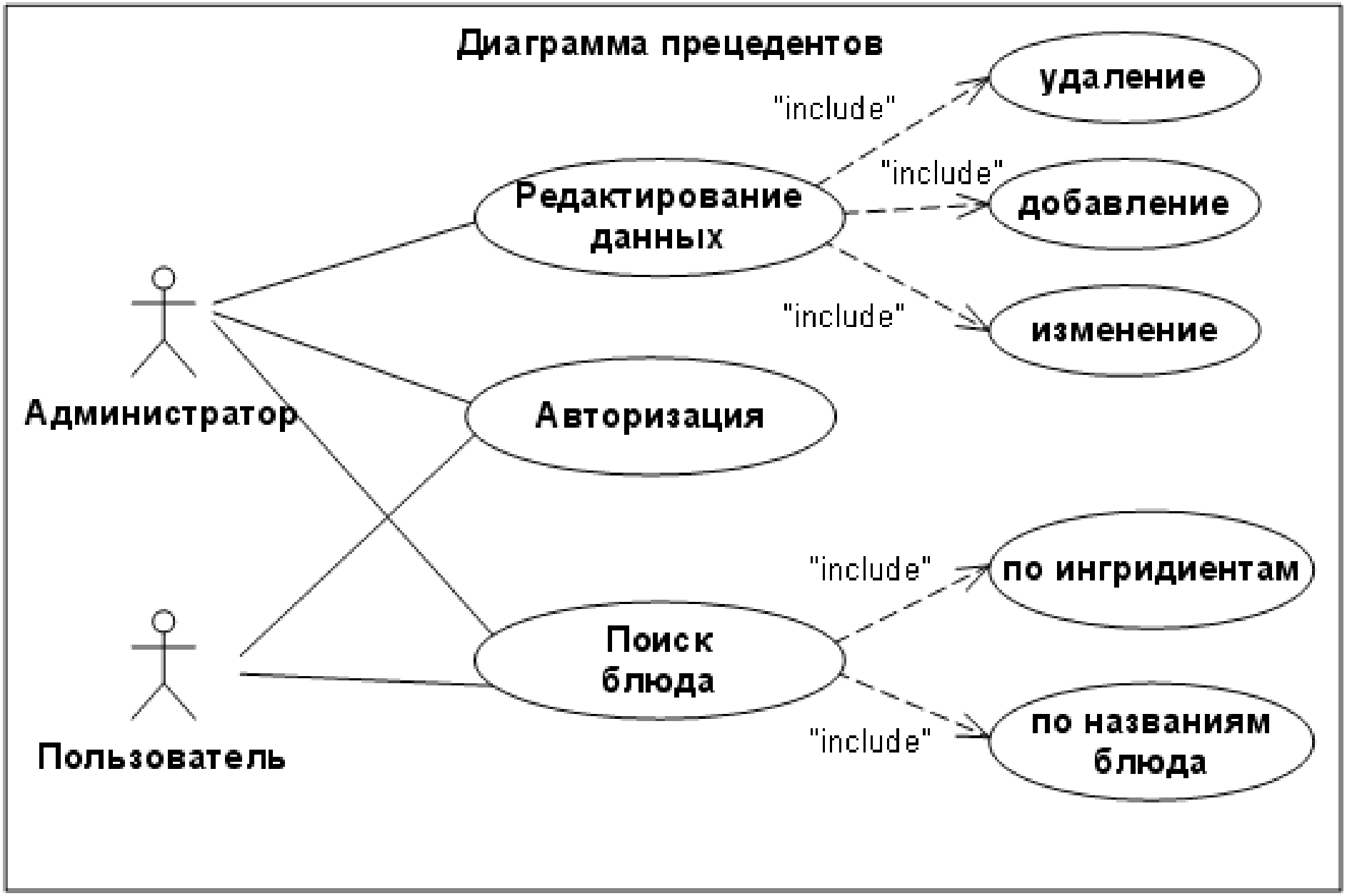
Следовательно не нужно ожидать соответствия между прецедентами и классами внутри системы.

Несмотря на то что в языке UML ничего не говорится о тексте прецедентов, именно текстовое содержание прецедентов является основной ценностью этой технологии.

Пример диаграммы прецедентов



Пример диаграммы прецедентов



Пример диаграммы прецедентов

